Scheme Recursion Processing

Introduction to Data Structures

Kyuseok Shim

ECE, SNU.

SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
Procedure Main()
begin
1. while (true)
2.     Command := GetCommand()
3.     InitializeTokenizer(command)
4.     root := Read()
5.     result := Eval(root)
6.     PrintResult(result, true)
end
```

SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
Procedure Preprocessing(newcommand)
begin
// newcommand is an empty string when this procedure is first called
while (token := GetNextToken()) is not empty
    if token is "define"
        newcommand := Concatenate(newcommand, "define")
        token := GetNextToken()
        if token is "("
            // (define (square x) ( * x x )) ==>
            // (define square (lambda (x) ( * x x )))
            token := GetNextToken()
            newcommand := Concatenate(newcommand, token,
                                      "(lambda(", Preprocessing(newcommand), ")" )
```

SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
Procedure Preprocessing()
…
  elseif token is "'"
      // '(a b c) ==> (quote (a b c))
    newcommand := Concatenate(newcommand, "(quote")
    number_of_left_paren := 0
    do
      token := GetNextToken()
      newcommand := Concatenate(newcommand, token)
      if token is "("
        number_of_left_paren := number_of_left_paren+1
      elseif token is ")"
        number_of_left_paren := number_of_left_paren−1
    while (number_of_left_paren>0)
    newcommand := Concatenate(newcommand, ")" )
  else newcommand := Concatenate(newcommand, token)
return newcommand
end
```

SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
Procedure Eval(root)
begin
1. tokenindex := GetHashValue(Memory[root].lchild)
2. if (token index = PLUS)
3.    return GetHashValue(GetVal(Eval(Memory[Memory[root].rchild].lchild))
4.                  + GetVal(Eval(Memory[Memory[Memory[root].rchild].rchild].lchild)))
…
11. elseif (token index = isEQ) // eq?
12.    return Eval(Memory[Memory[root].rchild].lchild
13.                  = Eval(Memory[Memory[Memory[root].rchild].rchild].lchild)
14. elseif (token index = isEQUAL) // equal?
15.    return CheckStructure(Eval(Memory[Memory[root].rchild].lchild),
16.                              Eval(Memory[Memory[Memory[root].rchild].rchild].lchild))
```

SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
Procedure Eval(root)
…
elseif (token index = isNUMBER)
  if IsNumber(Eval(Memory[Memory[root].rchild].lchild)) is true
     return GetHashValue("#t")
  else return GetHashValue("#f")

elseif (token index = isSYMBOL)
   if result := EVAL(Memory[Memory[root].rchild].lchild) is true and IsNumber(result) is false
      return GetHashValue("#t")
   else return GetHashValue("#f")

elseif (token index = isNULL)
   if Memory[root].rchild is NIL or Eval(Memory[root].rchild) is NIL
      return GetHashValue("#t")
   else return GetHashValue("#f")
```

SCHEME INTERPRETER PSEUDO CODE (2nd Version)

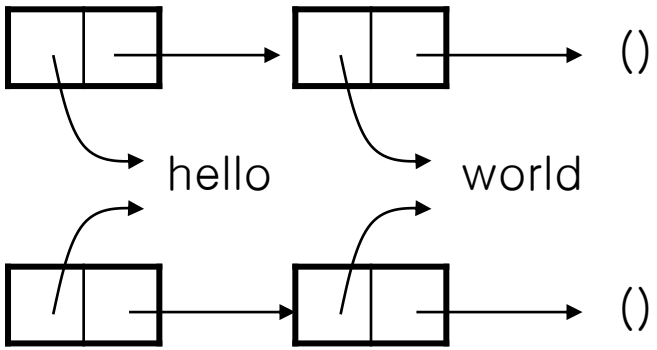
```
Procedure Eval(root)
…
elseif (token index = CONS)
  newmemory := Alloc()
  Memory[newmemory].lchild := Eval(Memory[Memory[root].rchild].lchild)
  Memory[newmemory].rchild := Eval(Memory[Memory[Memory].root].rchild].rchild].lchild)
  return newmemory

elseif (token index = COND)
  while Memory[Memory[root].rchild].rchild is not NIL
     root := Memory[root].rchild
     if (EVAL(Memory[Memory[root].lchild].lchild) = TRUE)
       return EVAL(Memory[Memory[root].lchild].rchild)
 if Memory[Memory[Memory[root].rchild].lchild].lchild is not ELSE
    Error()
 return Eval(Memory[Memory[Memory[Memory[root].rchild].lchild].rchild].lchild)
```

Notions of Equality (1)

- equal? : recursively checks whether its two arguments are lists with “equal” elements.
- eq? : checks whether its two arguments are identical pointers.

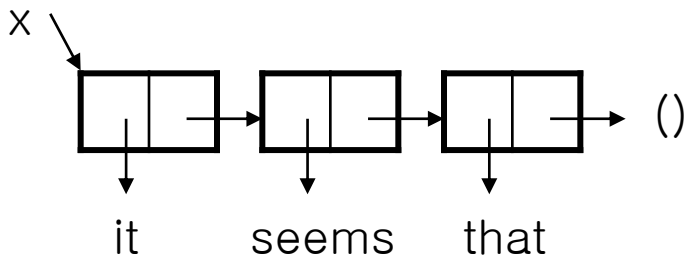
Notions of Equality (2)

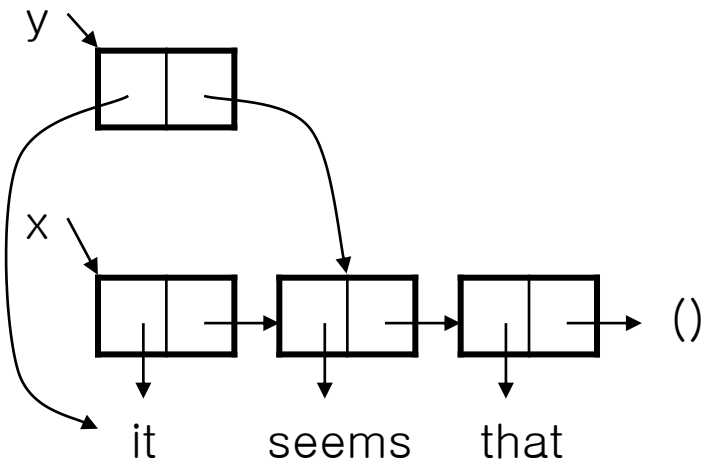
- (equal? ‘hello ‘hello) : #t
  (eq? ‘hello ‘hello) : #t
- (equal? ‘(hello world) ‘(hello world)) : #t
  (eq? ‘(hello world) ‘(hello world)) : #f



Notions of Equality (3)

- (define x ‘(it seems that))
- (define y (cons (car x) (cdr x)))
- (equal? x y) : #t
- (eq? x y) : #f





Allocation and Deallocation (1)

- Cells that are no longer in use have to be deallocated.
- A standard technique is to link cells on a list called a *free list*. (acts like a stack of cells)
  - pop : returns a freshly allocated cell.
  - push : returns a cell back onto the stack.
- Garbage collection returns cells to the free list automatically
  - without explicit instructions from a program.

Allocation and Deallocation (2)

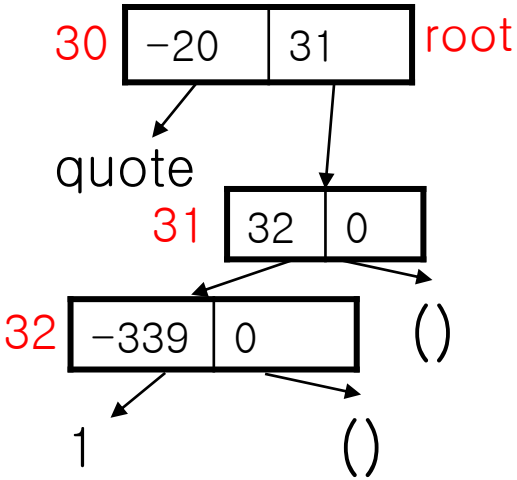
- Two approaches to deallocation of cells
  - Lazy approach : Wait until memory runs out and only then collect dead cells.
  - Eager approach : Each time a cell is reached, check whether the cell will be needed after the operation.

Allocation and Deallocation (3)

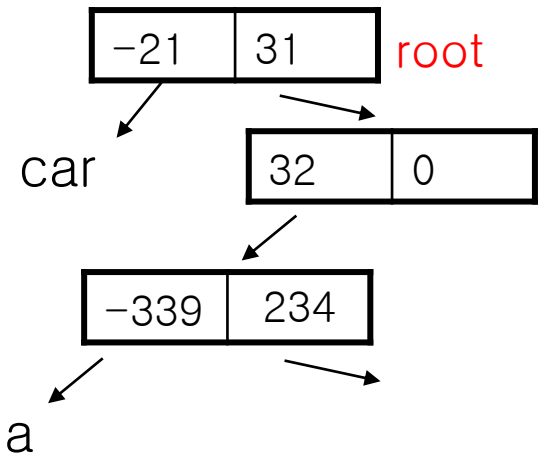
- A simple approach to garbage collection
  - Mark phase : Mark all the cells that can be reached by following pointers.
  - Sweep phase : Sweep through memory, looking for unmarked cells.

SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
Procedure Eval(root)
…
elseif (token index = CAR)
   return Memory[EVAL(Memory[Memory[root].rchild].lchild)].lchild

elseif (token index = CDR)
   return Memory[EVAL(Memory[Memory[root].rchild].lchild)].rchild

elseif (token index = DEFINE)
   if function define
      hashTable[Memory[Memory[root].rchild].lchild].pointer :=
                        Eval(Memory[Memory[Memory[root].rchild].rchild].lchild)
   else hashTable[Memory[Memory[root].rchild].lchild].pointer :=
                         EVAL(Memory[Memory[root].rchild].rchild)

   elseif (token index = QUOTE)
      return Memory[Memory[root].rchild].lchild
```



SCHEME INTERPRETER PSEUDO CODE (2nd Version)

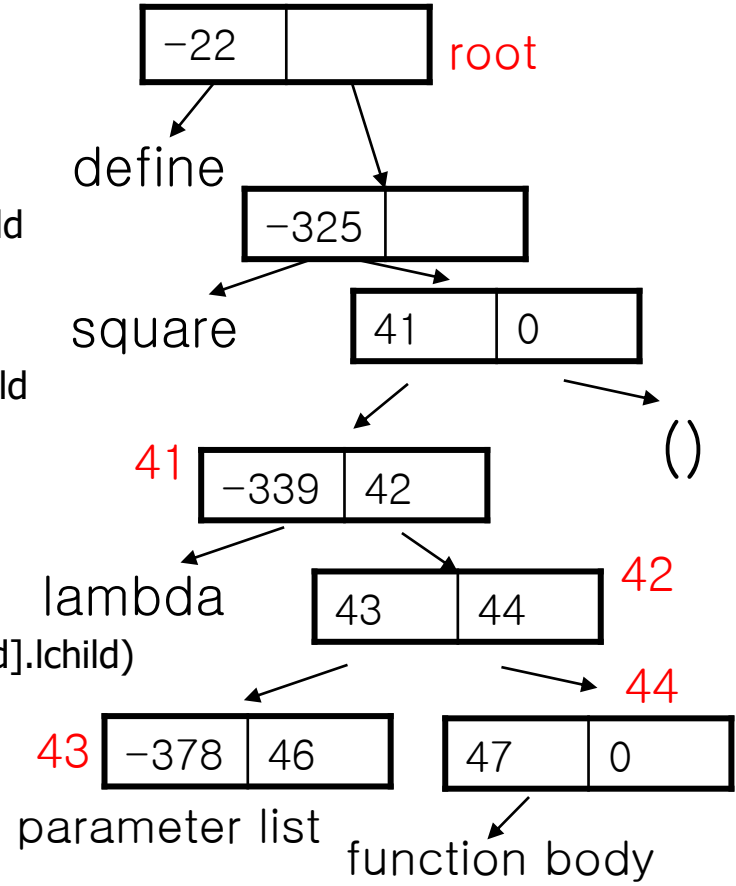



```
Procedure Eval(root)
…
elseif (token index = CAR)
  return Memory[EVAL(Memory[Memory[root].rchild].lchild)].lchild

elseif (token index = CDR)
  return Memory[EVAL(Memory[Memory[root].rchild].lchild)].rchild

elseif (token index = DEFINE)
  if function define
     hashTable[Memory[Memory[root].rchild].lchild].pointer :=
                    Eval(Memory[Memory[Memory[root].rchild].rchild].lchild)
  else hashTable[Memory[Memory[root].rchild].lchild].pointer :=
                     EVAL(Memory[Memory[root].rchild].rchild)

  elseif (token index = QUOTE)
     return Memory[Memory[root].rchild].lchild
```

()

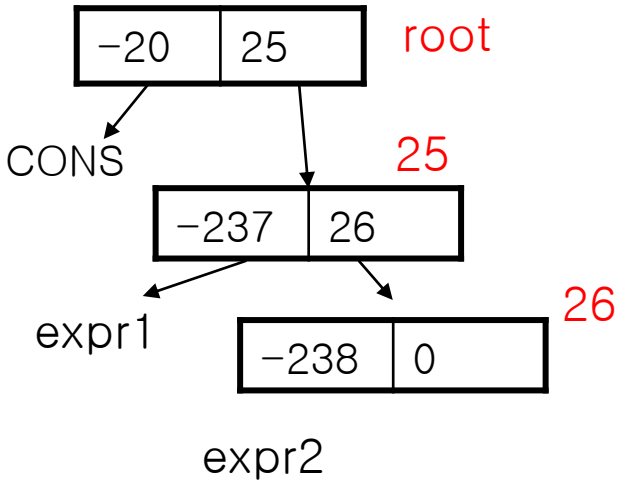
SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
Procedure Eval(root)
…
elseif (token index = CAR)
   return Memory[EVAL(Memory[Memory[root].rchild].lchild)].lchild

elseif (token index = CDR)
   return Memory[EVAL(Memory[Memory[root].rchild].lchild)].rchild

elseif (token index = DEFINE)
   if function define
      hashTable[Memory[Memory[root].rchild].lchild].pointer :=
                    Eval(Memory[Memory[Memory[root].rchild].rchild].lchild)
   else hashTable[Memory[Memory[root].rchild].lchild].pointer :=
                    EVAL(Memory[Memory[root].rchild].rchild)

   elseif (token index = QUOTE)
      return Memory[Memory[root].rchild].lchild
```



SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
Procedure Eval(root)
…
elseif (token index = CONS)
   newmemory := Alloc()
   Memory[newmemory].lchild := Eval(Memory[Memory[root].rchild].lchild)
   Memory[newmemory].rchild := Eval(Memory[Memory[Memory].root].rchild].rchild].lchild)
   return newmemory
```

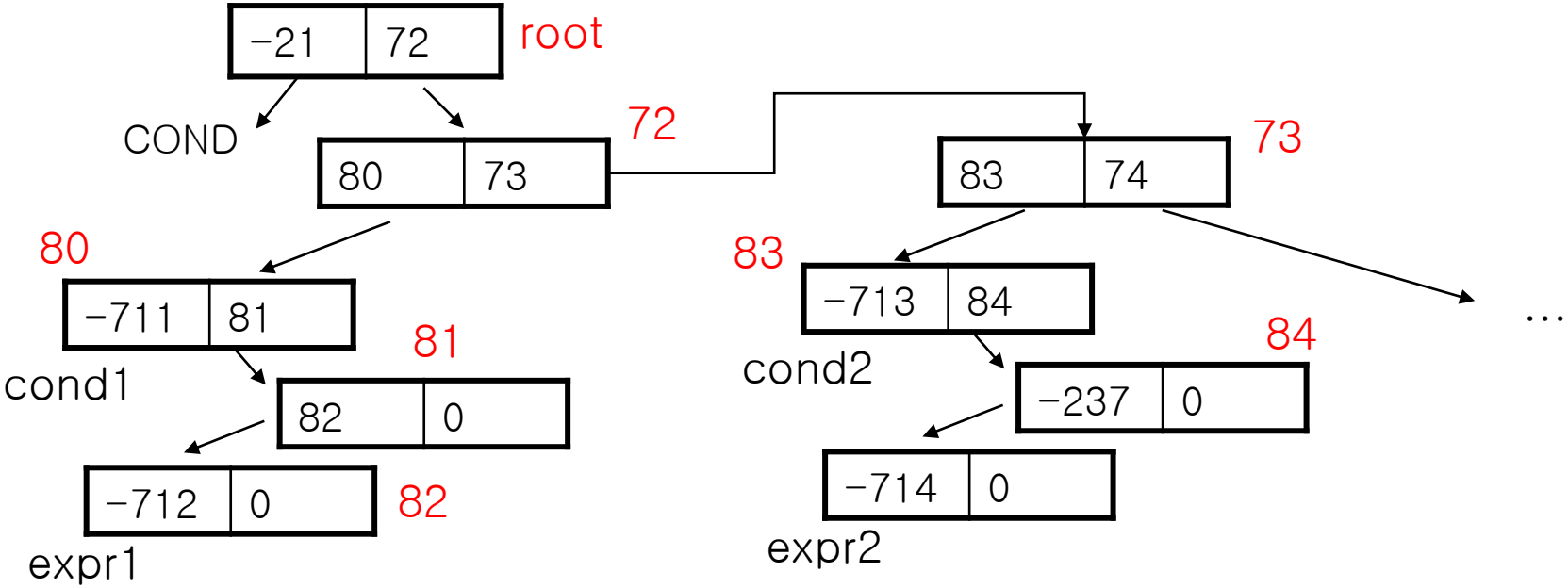


(CONS expr1 expr2)

SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
elseif (token index = COND)
  while Memory[Memory[root].rchild].rchild is not NIL
    root := Memory[root].rchild
    if (EVAL(Memory[Memory[root].lchild].lchild) = TRUE)
      return EVAL(Memory[Memory[root].lchild].rchild)
if Memory[Memory[Memory[root].rchild].lchild].lchild is not ELSE
  Error()
return Eval(Memory[Memory[Memory[Memory[root].rchild].lchild].rchild].lchild)
```

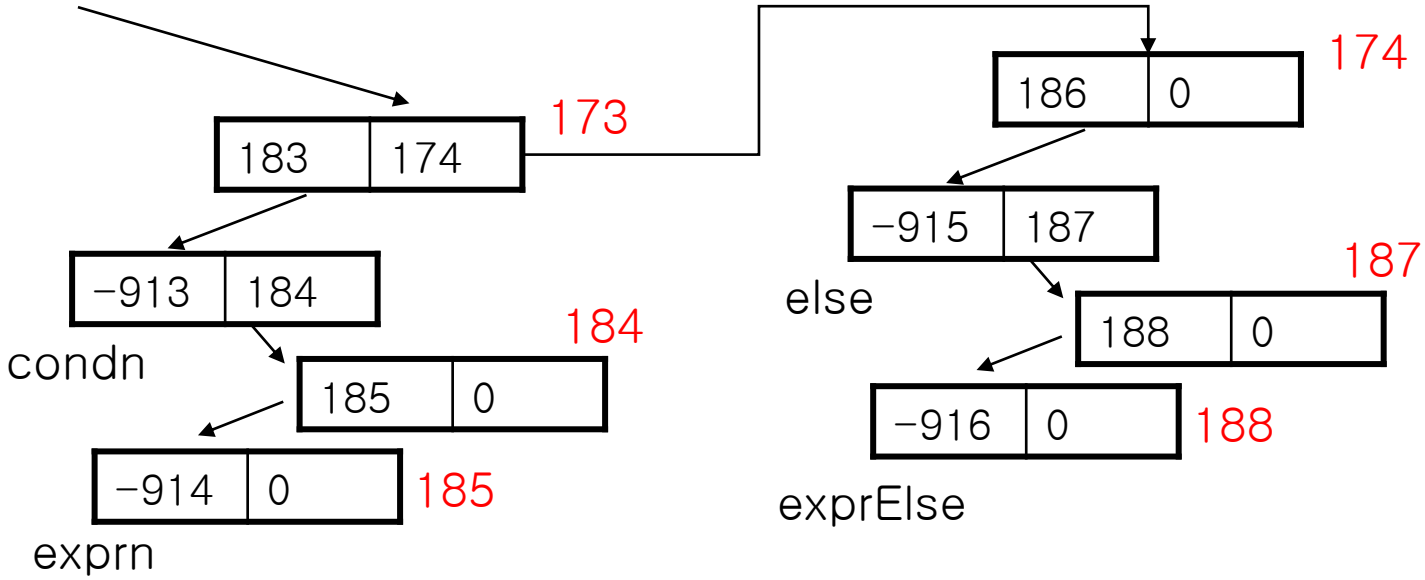
```
(COND ((cond1) (expr1))
      ((cond2) (expr2))
      ...
      ((condn) (exprn))
      ((else) (exprElse)))
```



SCHEME INTERPRETER PSEUDO CODE (2nd Version)

```
elseif (token index = COND)
  while Memory[Memory[root].rchild].rchild is not NIL
     root := Memory[root].rchild
     if (EVAL(Memory[Memory[root].lchild].lchild) = TRUE)
       return EVAL(Memory[Memory[root].lchild].rchild)
  if Memory[Memory[Memory[root].rchild].lchild].lchild is not ELSE
    Error()
  return Eval(Memory[Memory[Memory[Memory[root].rchild].lchild].rchild].lchild)
```

```
(COND ((cond1) (expr1))
      ((cond2) (expr2))
      ...
      ((condn) (exprn))
      ((else) (exprElse)))
```

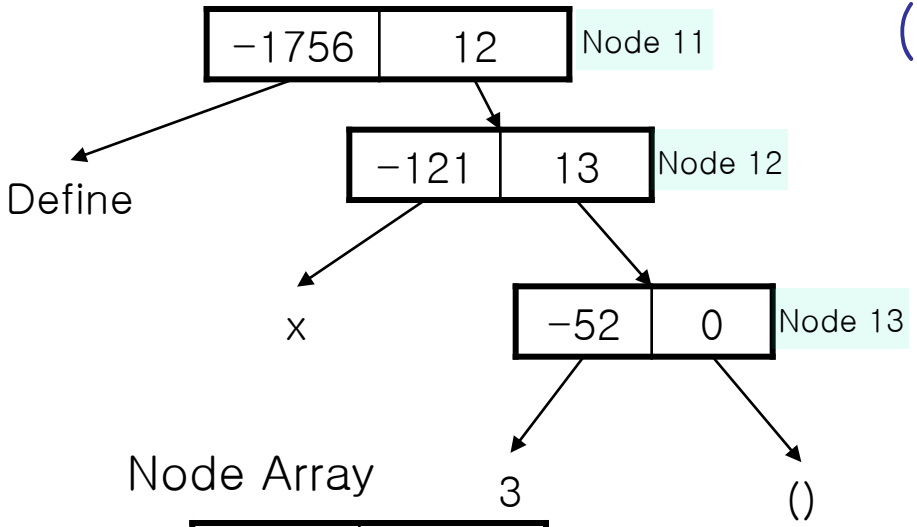


Parse Tree & Node Array

- Build the parse tree with the node array when a command input comes
  - Read the input token by token
  - Make an new node and attach it to the tree
  - When we make a new node, allot it from the array
- Each node stores the indices of its left child and right child

Parse Tree & Node Array Example (1)

(define x 3)



Node Array

Array Index	Left	Right
…		
11	−1756	12
12	−121	13
13	−52	0

Symbol Hash Table

Hash Value	Symbol	Link of Value
…		
−52	3	NULL
…		
−121	x	NULL
…		
−1756	define	NULL
…		

Parse Tree & Node Array Example (2)

- **Input: (define x 3)**
  - Read the token '(' and allot the new root node (node 11) from the array.
  - Read the token ‘define’. Store the hash table index (= -1756) of 'define' as the left child index of node 11 and the node id (= 12) of the newly allotted node as the right child index of node 11.

Parse Tree & Node Array Example (3)

- Read the token 'x' and store the hash table index (= -121) of ‘x’ as the left child index of node 12. Then allot the new node and store the id (= 13) of it as the right child index of node 12.
- Read the next token ‘3’ and store the hash table index (= -52) of ‘3’ as the left child index of node 13. Since next token is ')', store null index (= 0) as the right child index of node 13 without allotting a new node. .

Symbol Table

- In Scheme, Symbol Table stores all “meaningful words”.
  - Built-in words, numbers, function names, symbols, etc.
  - Hash table should hold words and links for their contents.
- For fast search and retrieve, we’ll use hash table as a symbol table.
  - Use negative numbers for hash entries as its indices.
  - In the other hand, use positive numbers for entries of the node array

Symbol Table Example

Hash Value	Symbol	Link of Value
-1	"+"	NULL
-2	"car"	NULL
…		
-52	"3"	NULL
…		
-121	"x"	-52
…		
-3285	"list"	20
…		
-3501	"func"	25

- Assume we have a hash function "f" from each symbol to hash value.
- We assign some "special" region for "built-in words".
  - f("+") = -1
- Hash function example
  - f("3") = -52, "3" is a number.
  - f("x") = -121, "x" is a variable.
  - f("list") = -3285, "list" is a list. List have a link for list data.
  - f("func") = -3501, "func" is a function. Function also have a link for function contents.

Collision Resolution (1)

- Two different words can have same hash value.
  - Hash table can store one element for one hash value. So, “collision problem” may happen.
- Here, we use Open Addressing Policy for hash table.
  - Resolve collision problem by using the first empty element from hash value.
  - If f(“3”)=−231 and f(“list”)=−231, the latter entry uses −232 as hash value.

Collision Resolution (2)

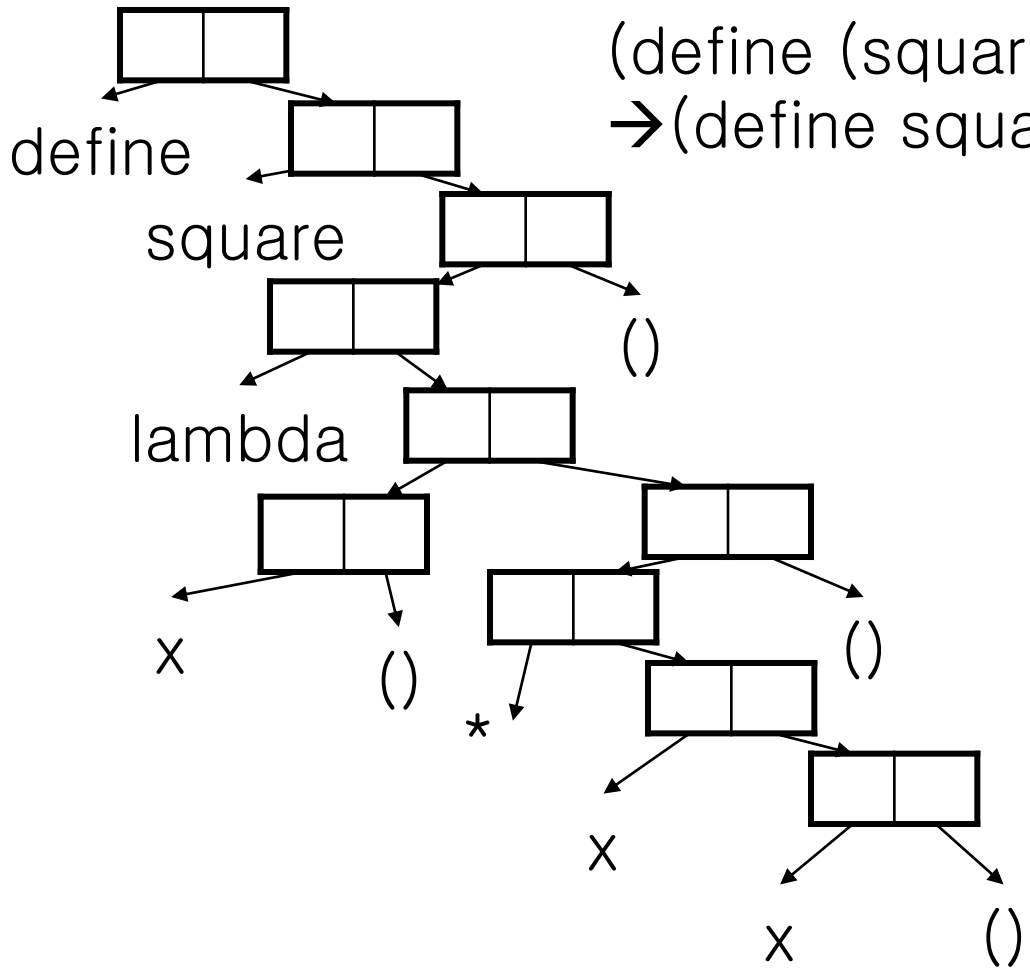
- Open Addressing has some weak points.
  - May make clusters, so search will be inefficient.
  - Very difficult to delete clustered entry.
- Here, We assume no delete operation and we have enough memory, so just use Open Addressing Policy.

Hash Value	Symbol	Link of Value
…	…	…
-231	"3"	NULL
-232	"list"	53
…	…	…

Example

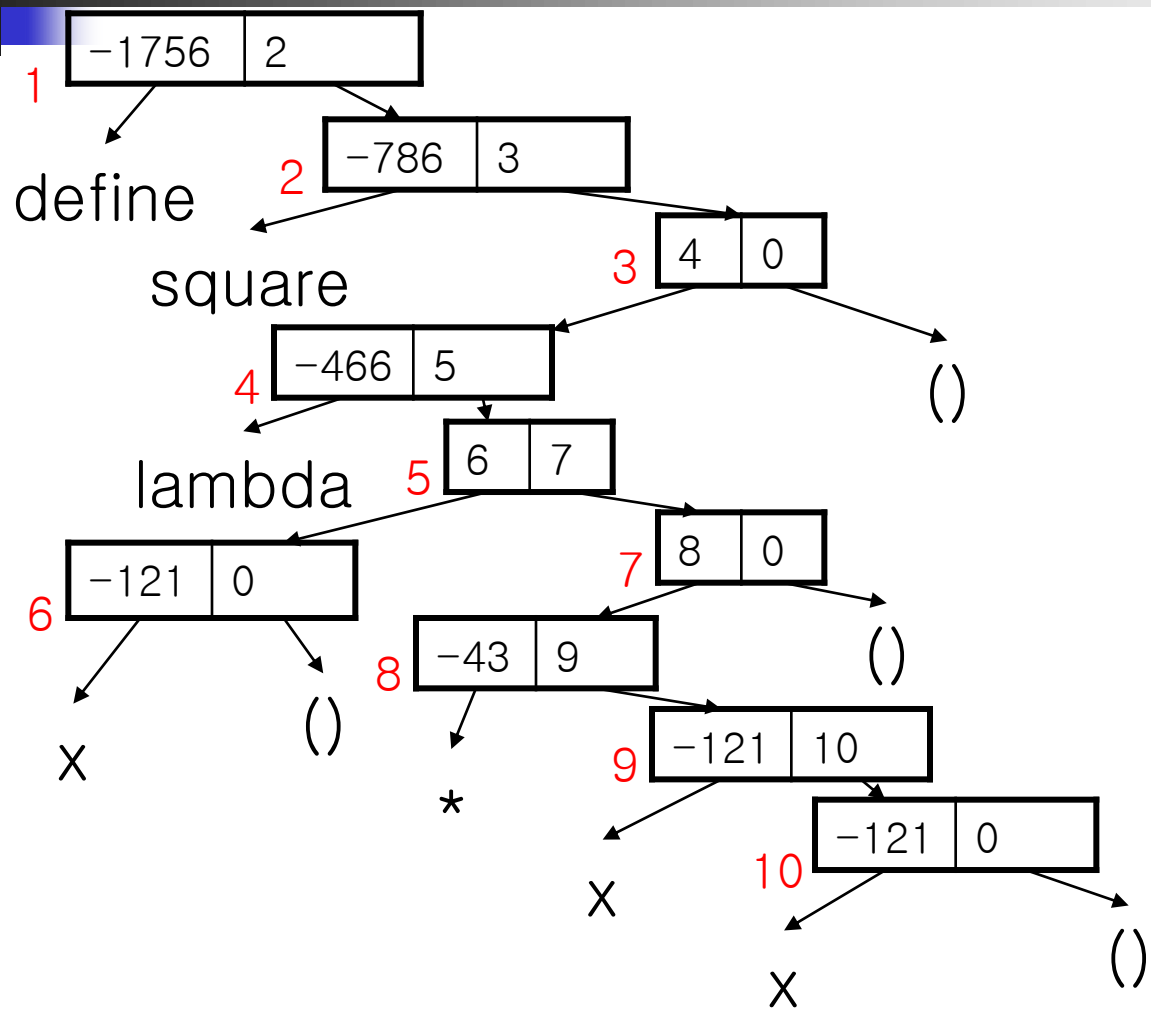
- Read & evaluate the following commands
  - (define (square x) (* x x))
  - (define x 3)
  - (square 6)

Convert non lambda form to lambda form



(define (square x) (* x x))
→(define square (lambda (x) (* x x)))

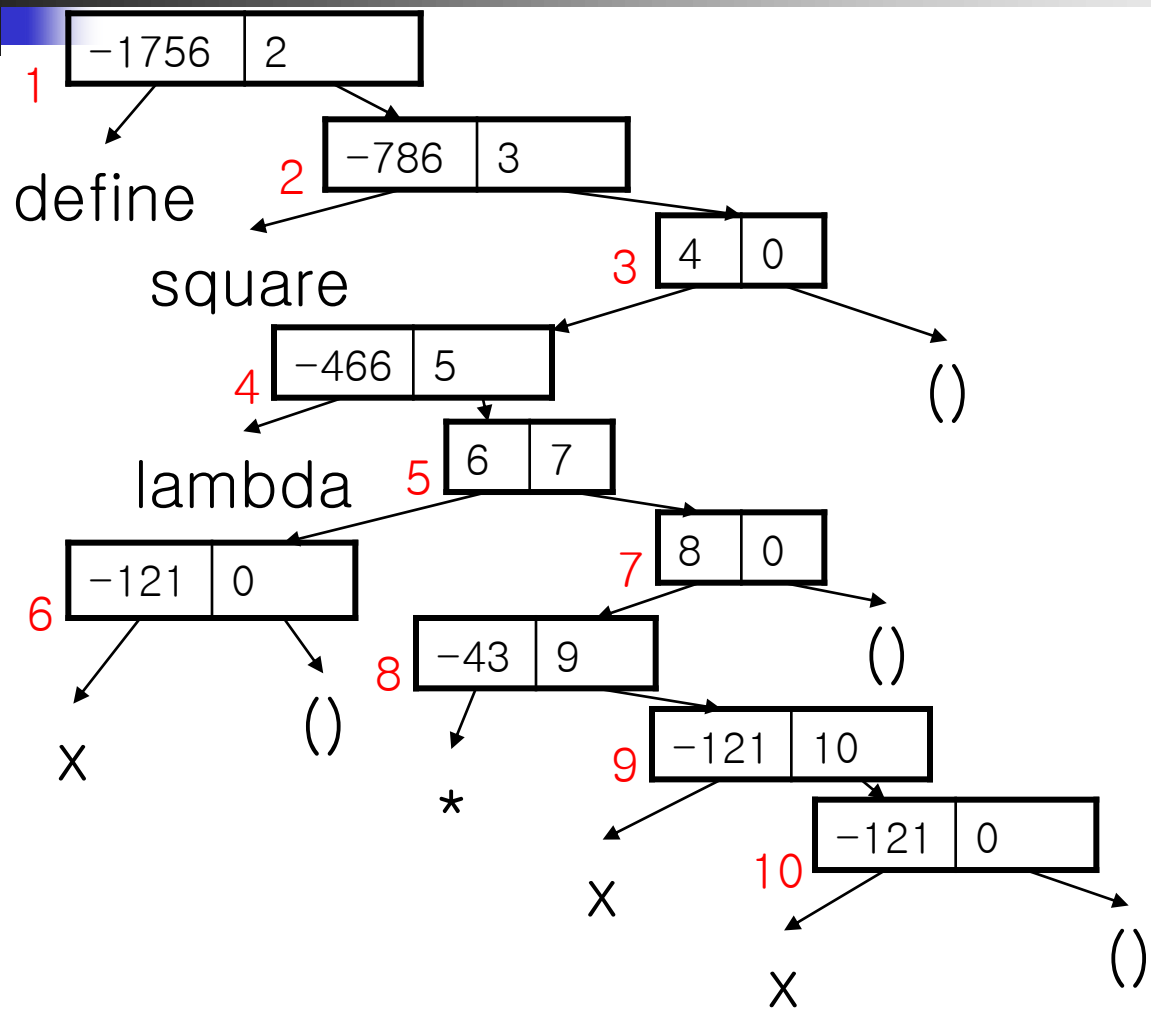
Parse Tree of (define square (lambda (x) (* x x)))



Node Array

Node ID	Left	Right
1	−1756	2
2	−786	3
3	4	0
4	−466	5
5	6	7
6	−121	0
7	8	0
8	−43	9
9	−121	10
10	−121	0

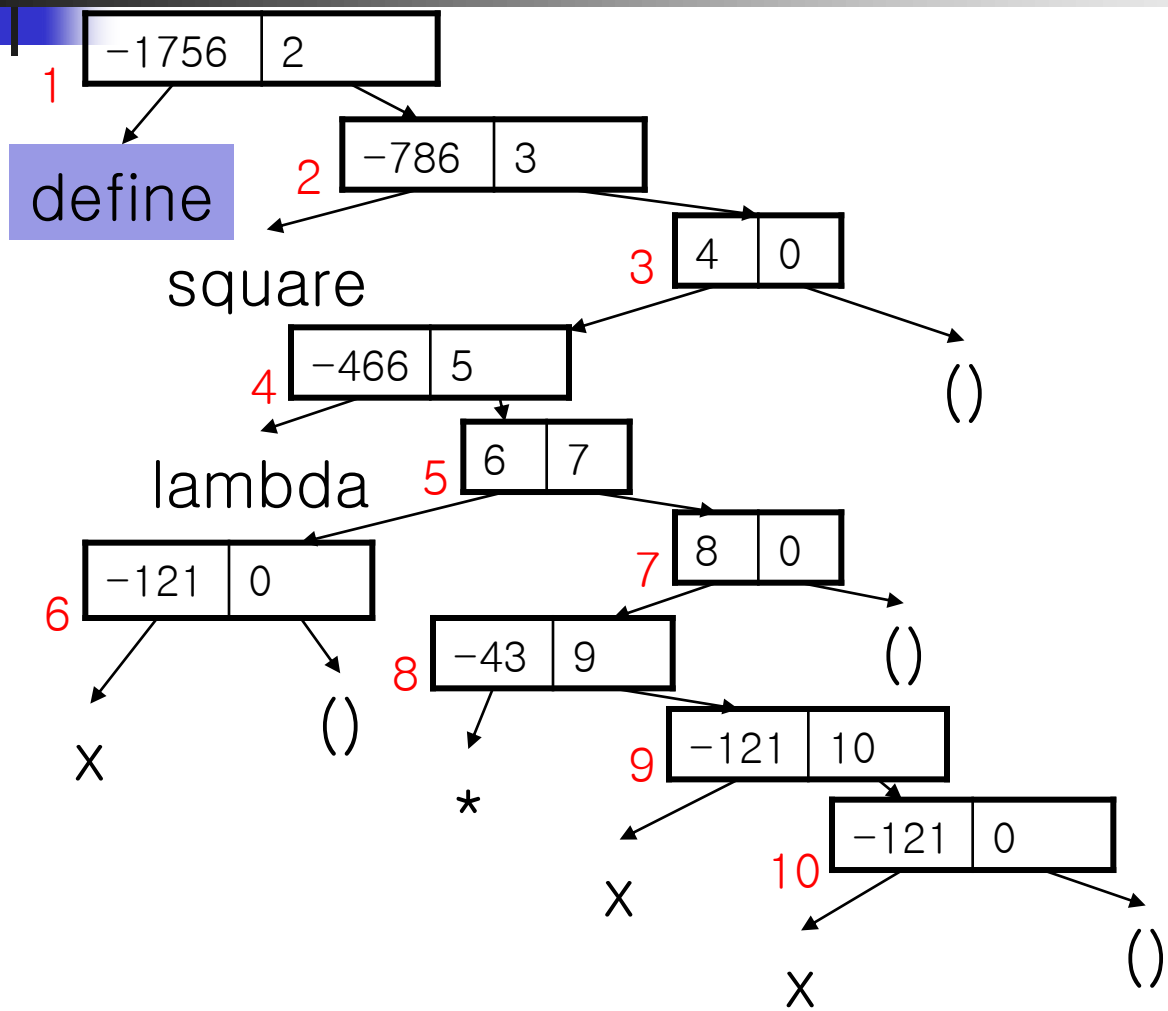
Parse Tree of (define square (lambda (x) (* x x)))



Hash Table

Hash Value	Symbol	Link of Value
…		
−43	*	
…		
−121	X	
…		
−466	Lambda	
…		
−786	square	
…		
−1756	Define	
…		

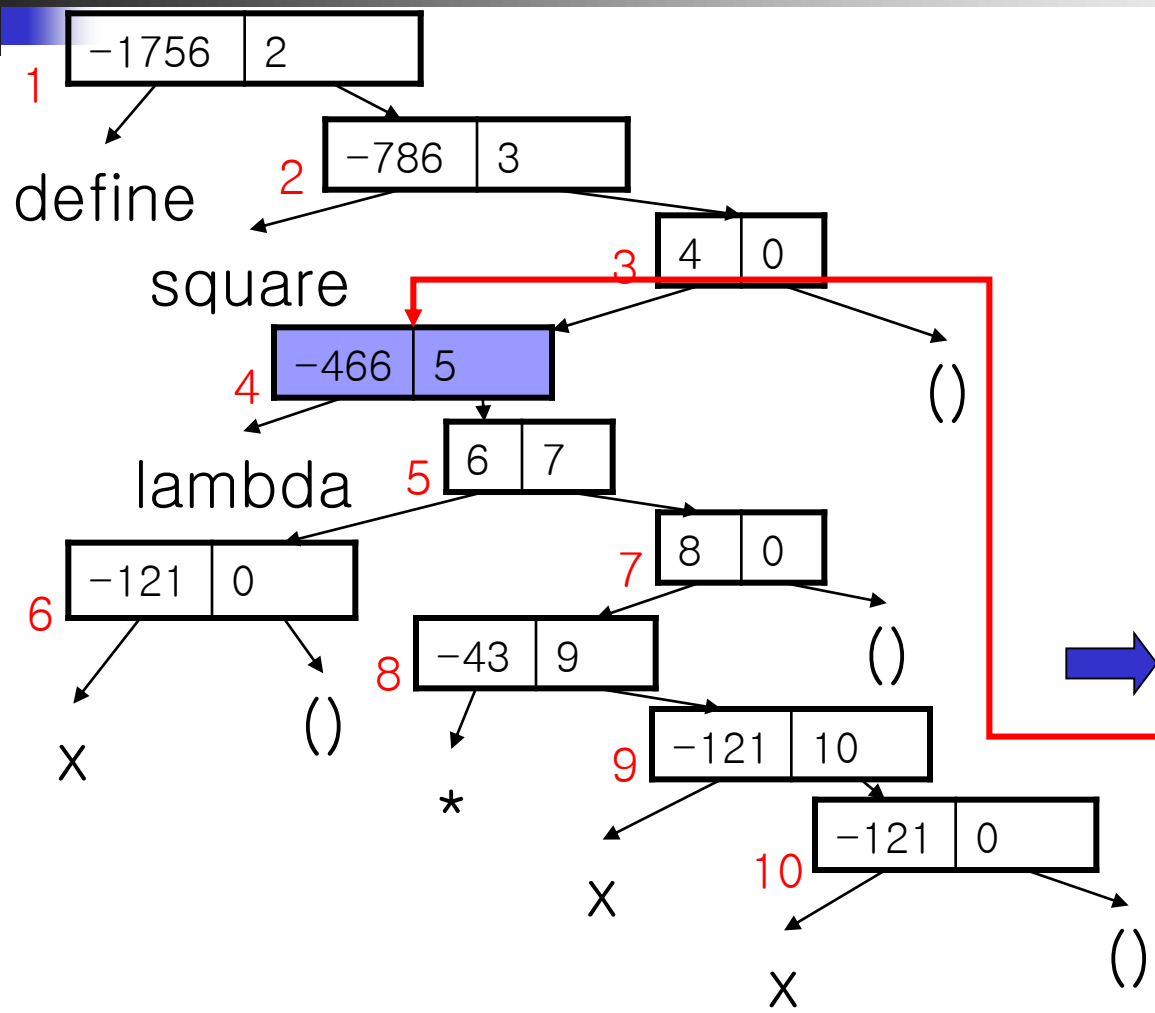
Evaluation of (define square (lambda (x) (* x x)))



Hash Table

Hash Value	Symbol	Link of Value
…		
−43	*	
…		
−121	X	
…		
−466	Lambda	
…		
−786	square	
…		
−1756	Define	
…		

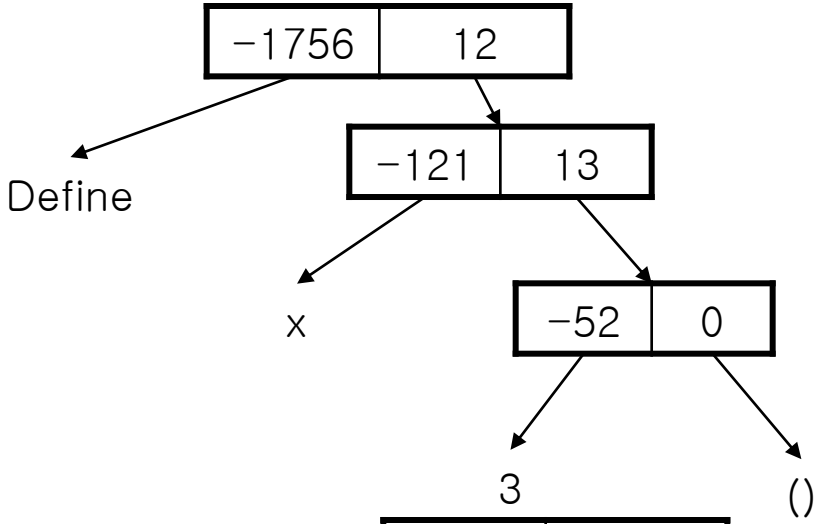
Evaluation of (define square (lambda (x) (* x x)))



Hash Table

Hash Value	Symbol	Link of Value
...		
−43	*	
...		
−121	X	
...		
−466	Lambda	
...		
−786	square	4
−1756	Define	
...		

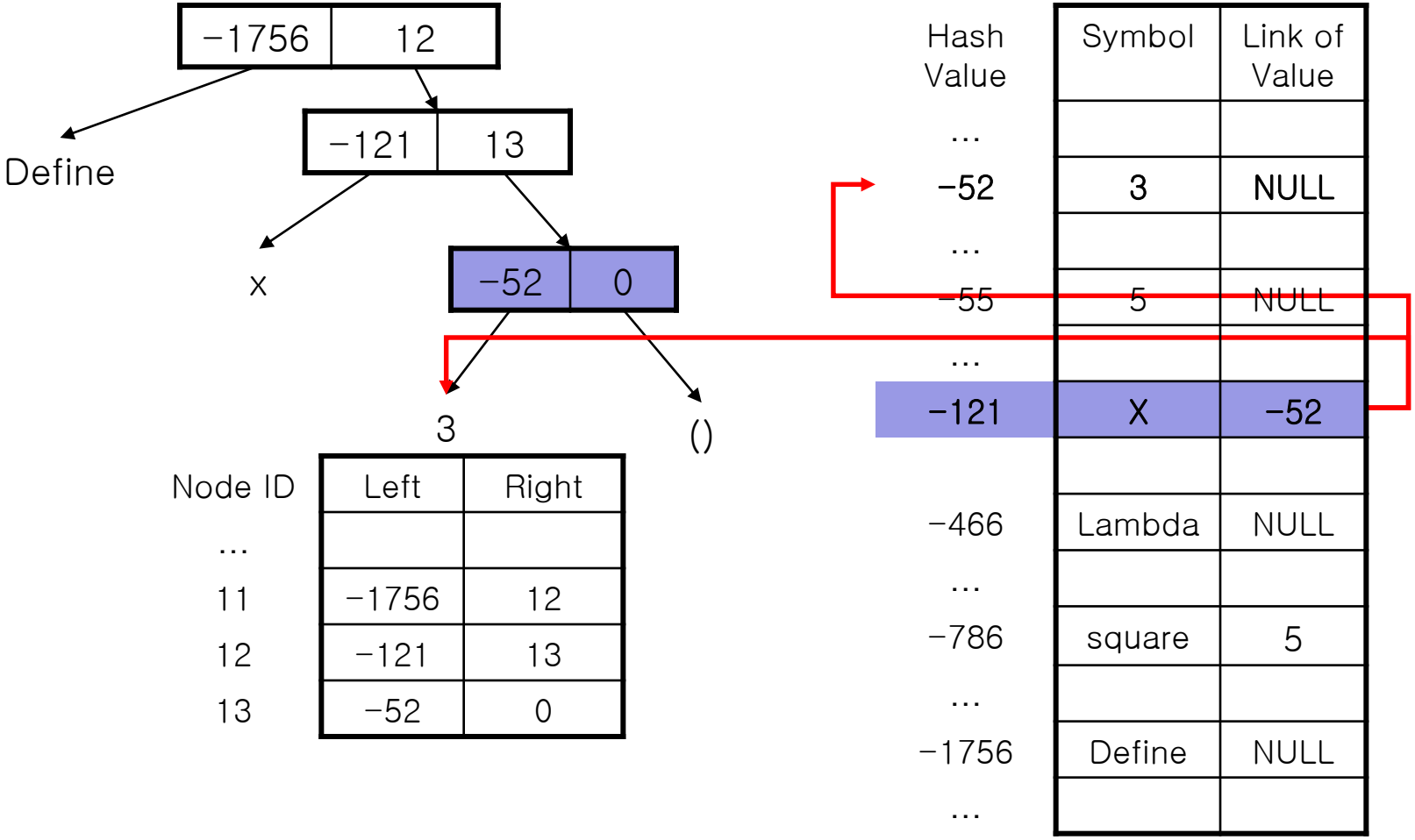
Parse Tree of (define x 3)



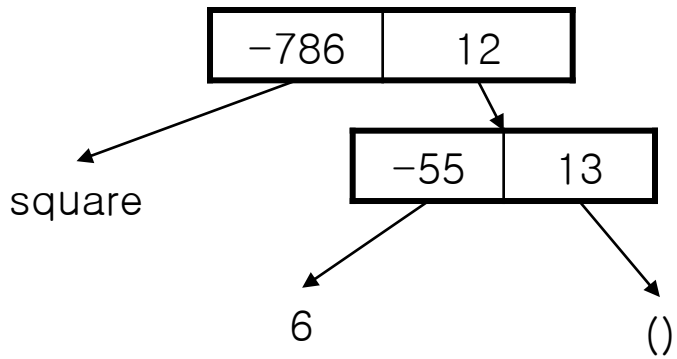
Node ID	Left	Right
…		
11	−1756	12
12	−121	13
13	−52	0

Hash Value	Symbol	Link of Value
…		
−43	*	NULL
…		
−52	3	NULL
…		
−121	X	NULL
..		
−466	Lambda	NULL
…		
−786	square	5
…		
−1756	Define	NULL
…		

Evaluation of (define x 3)



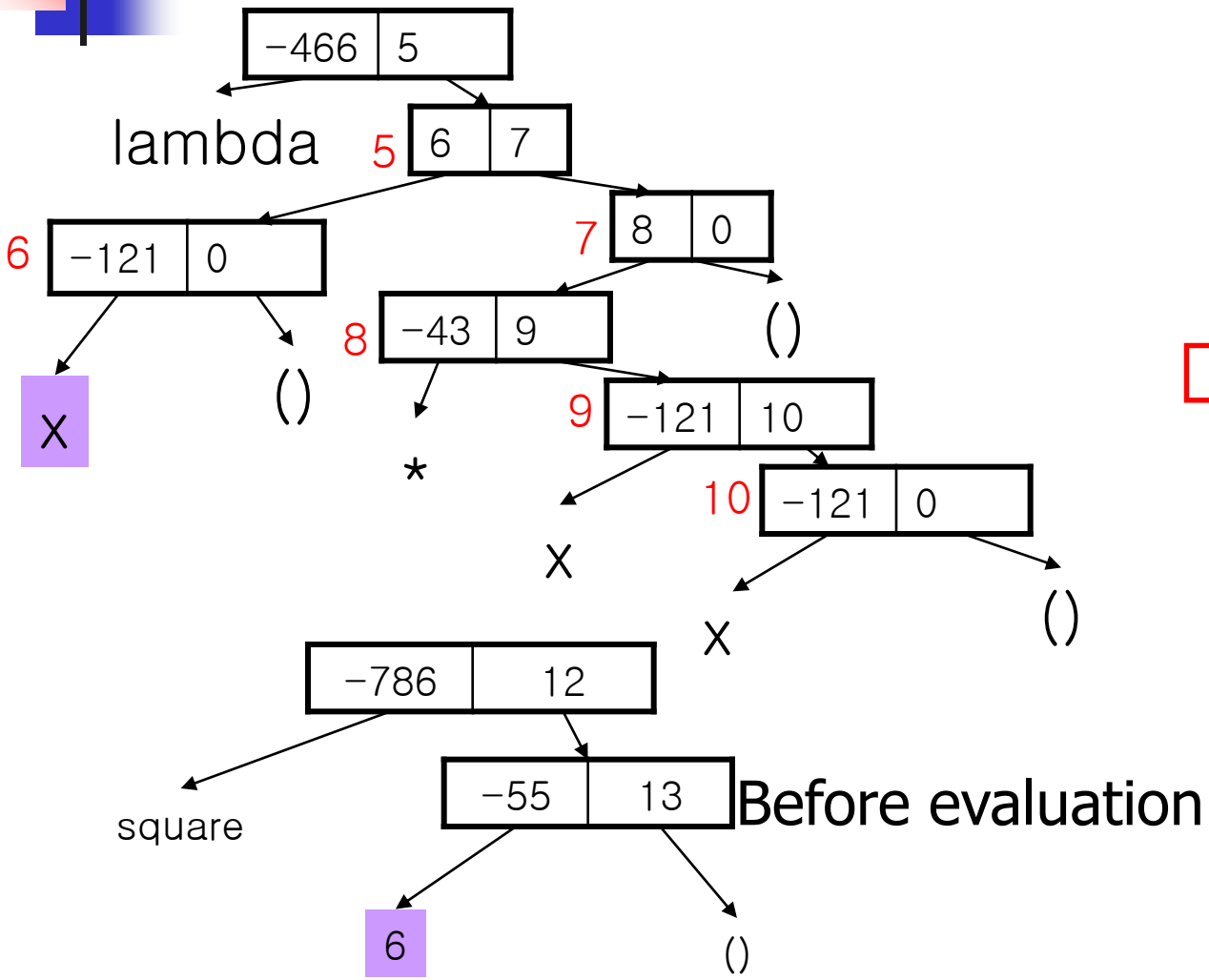
Parse Tree of (square 6)



Node ID	Left	Right
…		
14	−786	12
15	−55	13

Hash Value	Symbol	Link of Value
…		
−52	3	NULL
…		
−55	6	NULL
…		
−121	X	−52
…		
−466	Lambda	NULL
…		
−786	square	5
…		
−1756	Define	NULL
…		

Evaluation of (square 6)

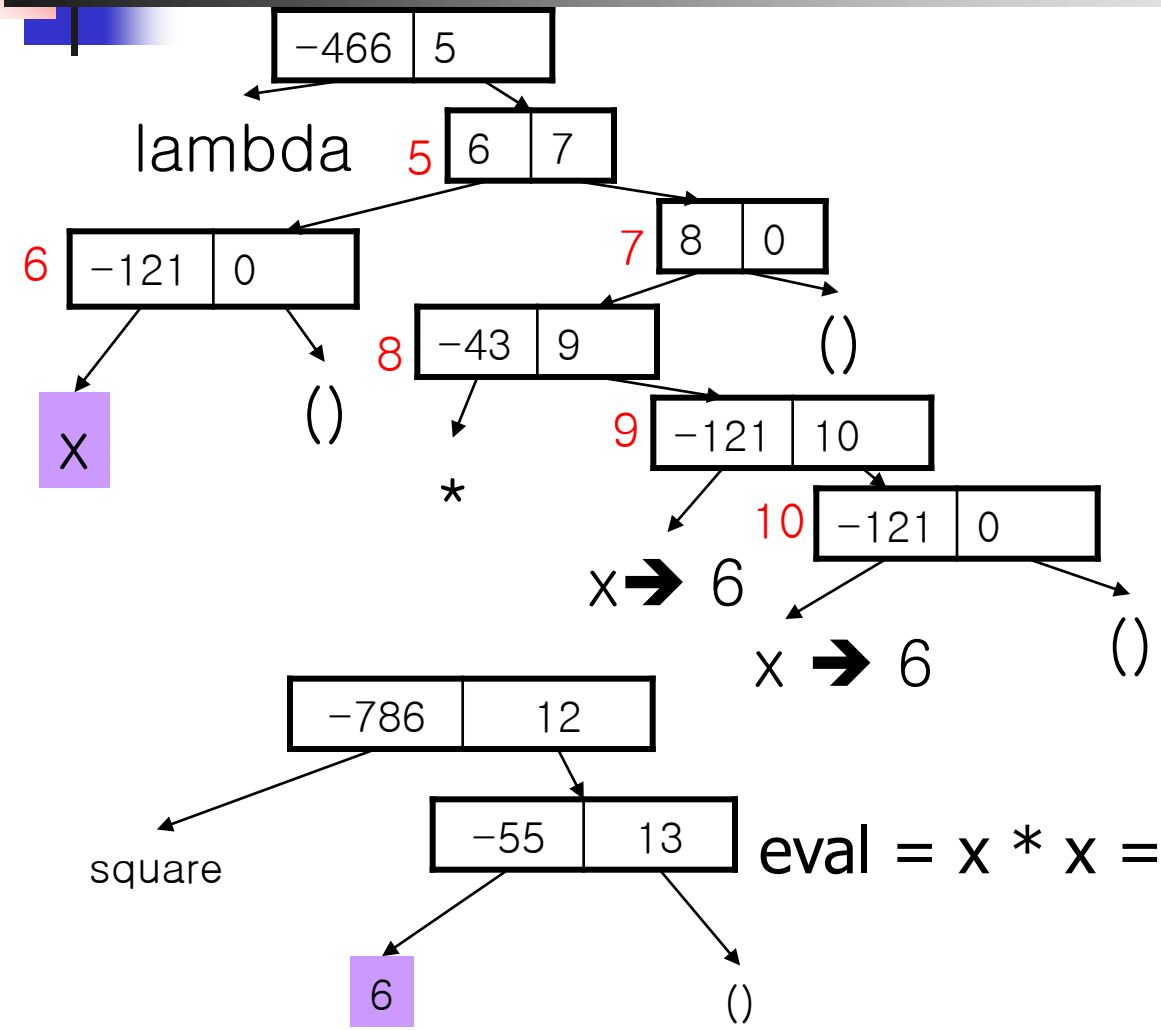


Hash Value	Symbol	Link of Value
…		
−52	3	NULL
…		
−55	6	NULL
…		
−121	X	−55
…		
−786	square	5
…		

Stack

−52

Evaluation of (square 6)



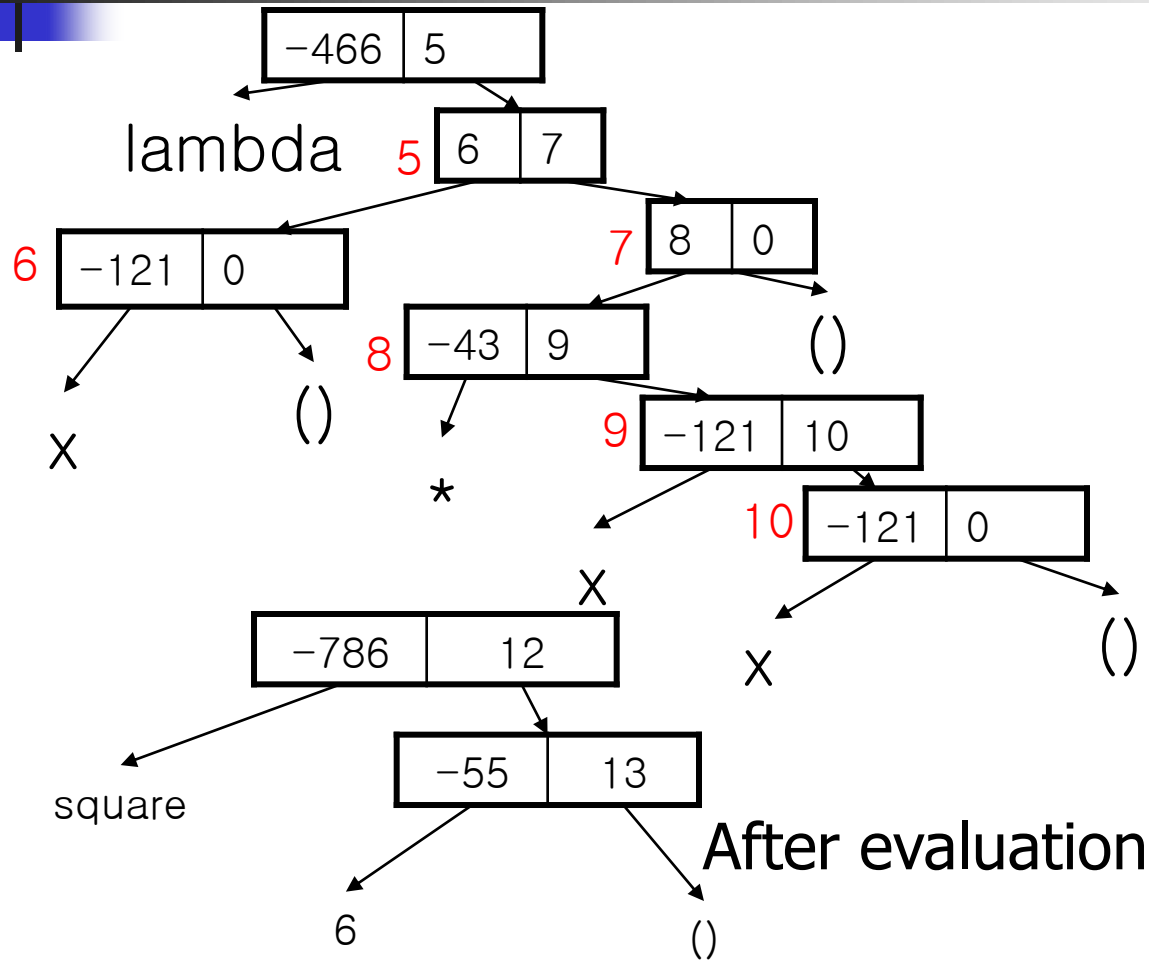
eval = x * x = 6 * 6 = 36

Hash Value	Symbol	Link of Value
…		
−52	3	NULL
…		
−55	6	NULL
…		
−121	X	−55
…		
−786	square	5
…		

Stack

−52

Evaluation of (square 6)



After evaluation

Hash Value	Symbol	Link of Value
...		
−52	3	NULL
−55	6	NULL
...		
−121	X	−52
...		
−786	square	5
...		

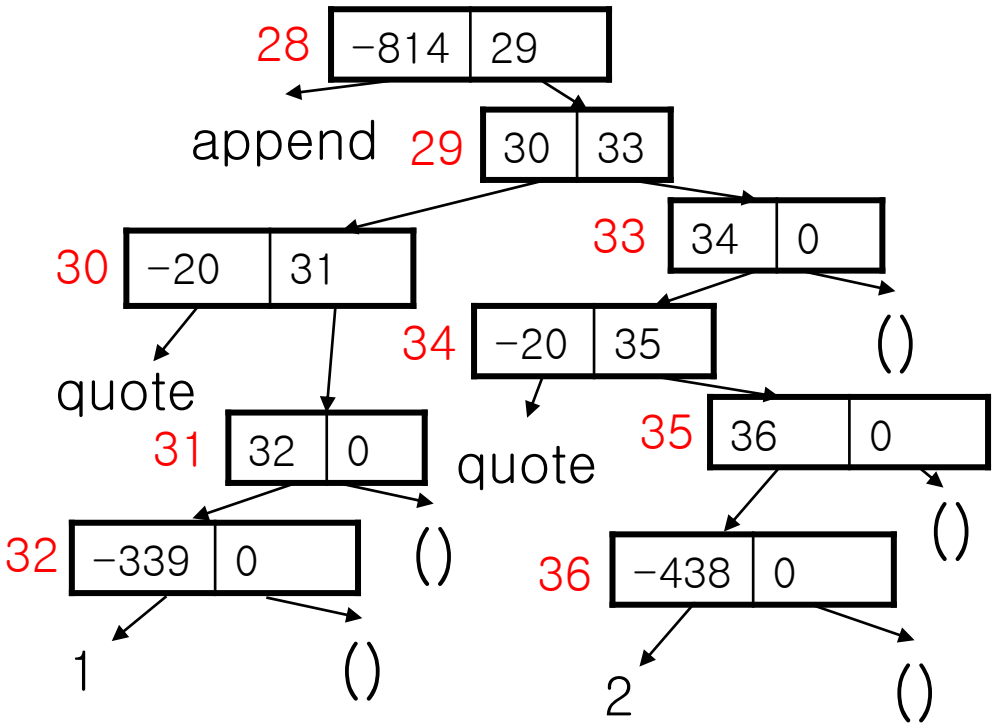
Stack

Append a list to another list

```
(define (append L R)
  (cond ((null? L) R)
    (else (cons (car L) (append (cdr L) R)))))
```

- (append '(1) '(2)) : (1 2)

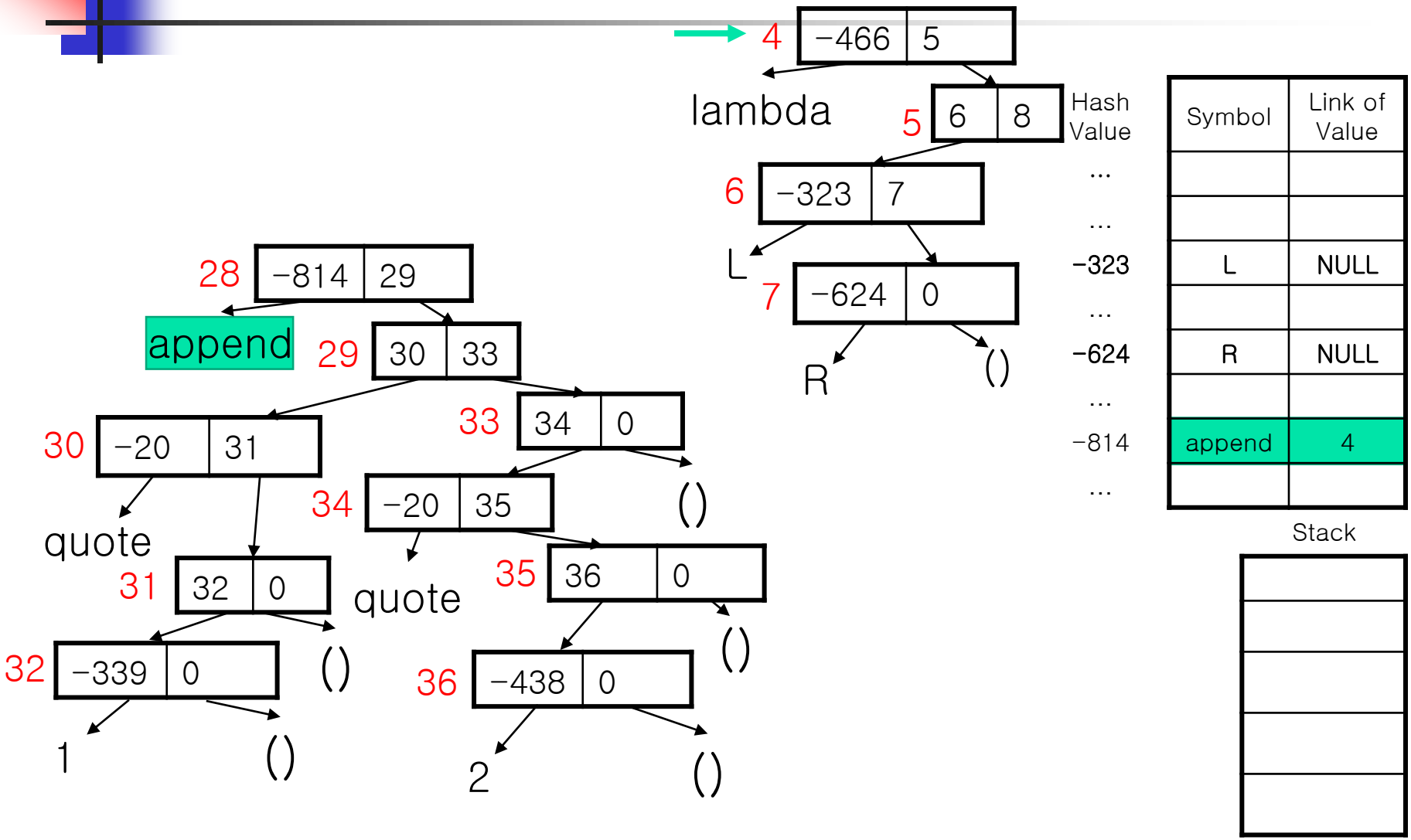
Evaluation of (append '(1) '(2))



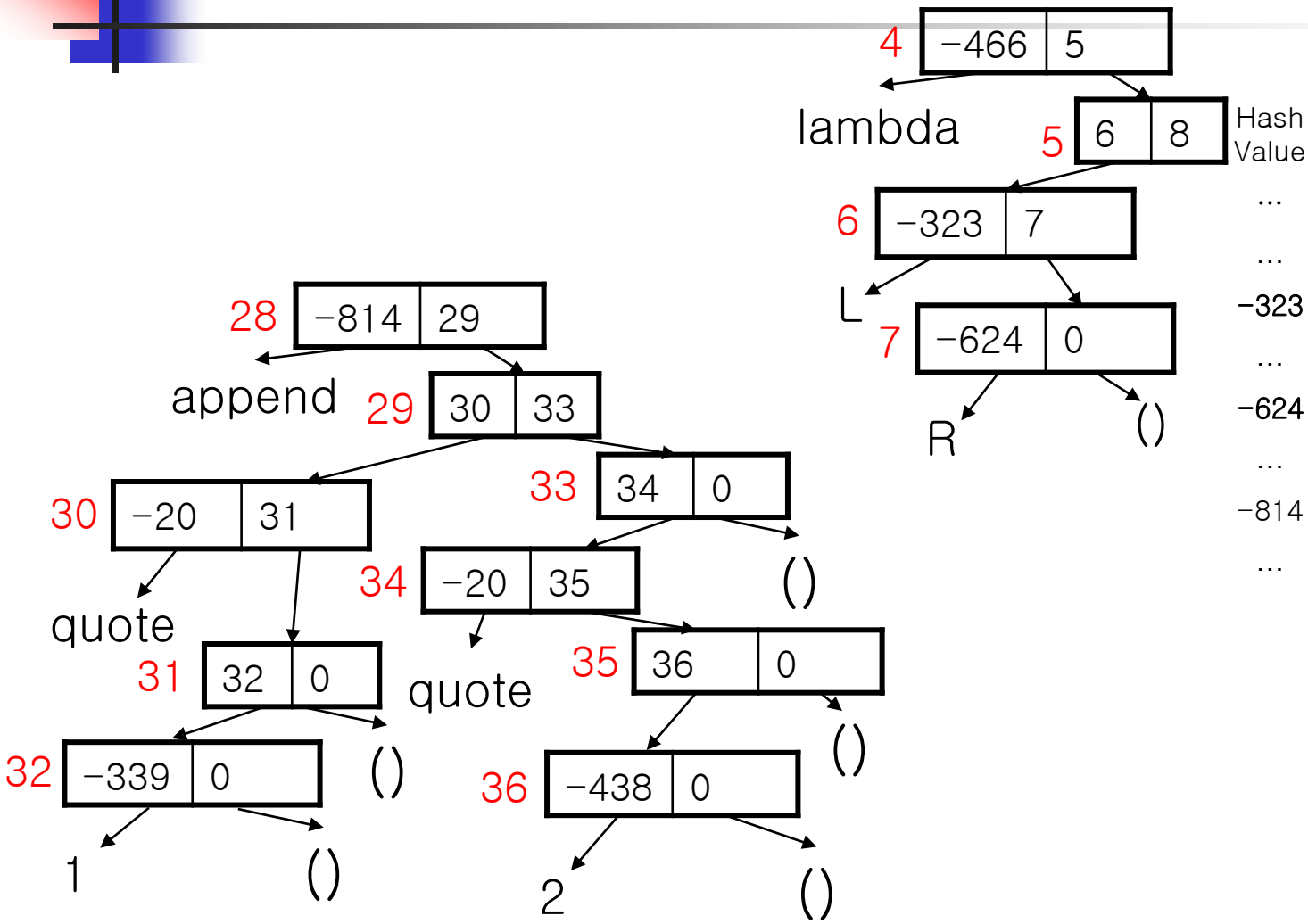
Hash Value	Symbol	Link of Value
…		
…		
-323	L	NULL
…		
-624	R	NULL
…		
-814	append	4
…		

Stack

Evaluation of (append '(1) '(2))



Evaluation of (append '(1) '(2))

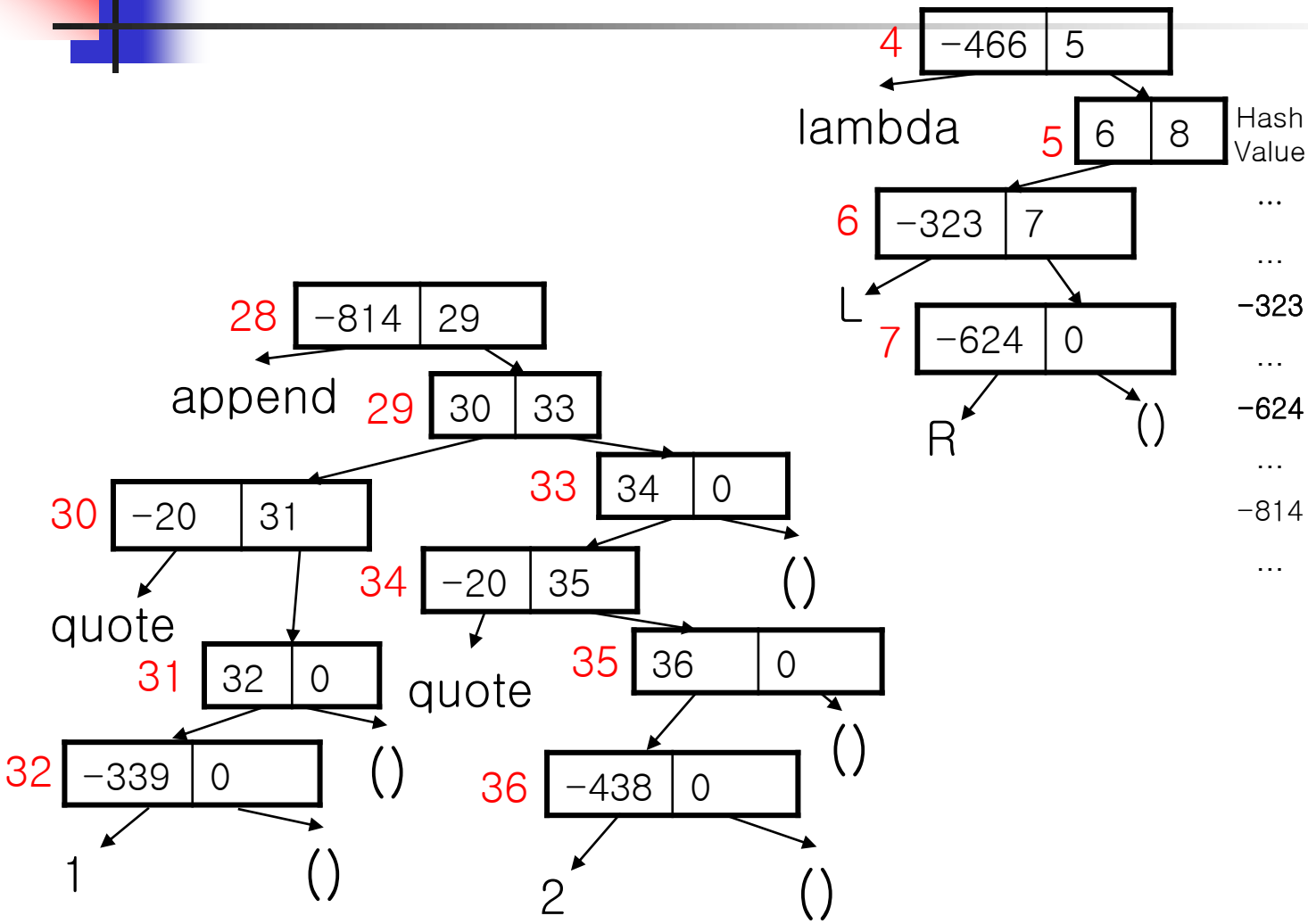


Symbol	Link of Value
L	32
R	NULL
append	4

Stack

L : NULL

Evaluation of (append '(1) '(2))



Symbol	Link of Value
L	32
R	36
append	4

Stack

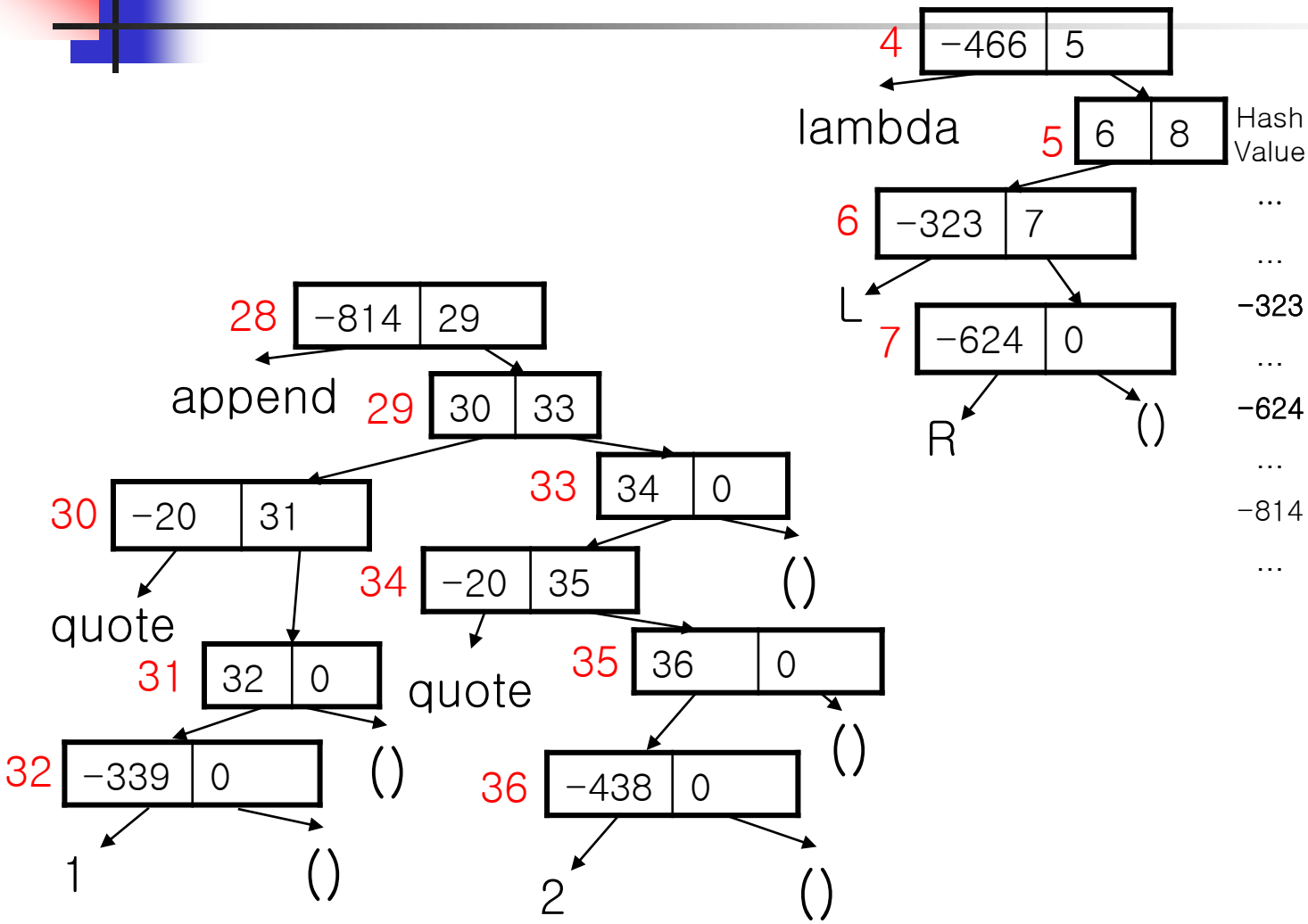
R : NULL
L : NULL

Append a list to another list

```
(define (append L R)
  (cond ((null? L) R)
    (else (cons (car L) (append (cdr L) R)))))
```

- (append '(1) '(2)) : (1 2)

Evaluation of (append '(1) '(2))

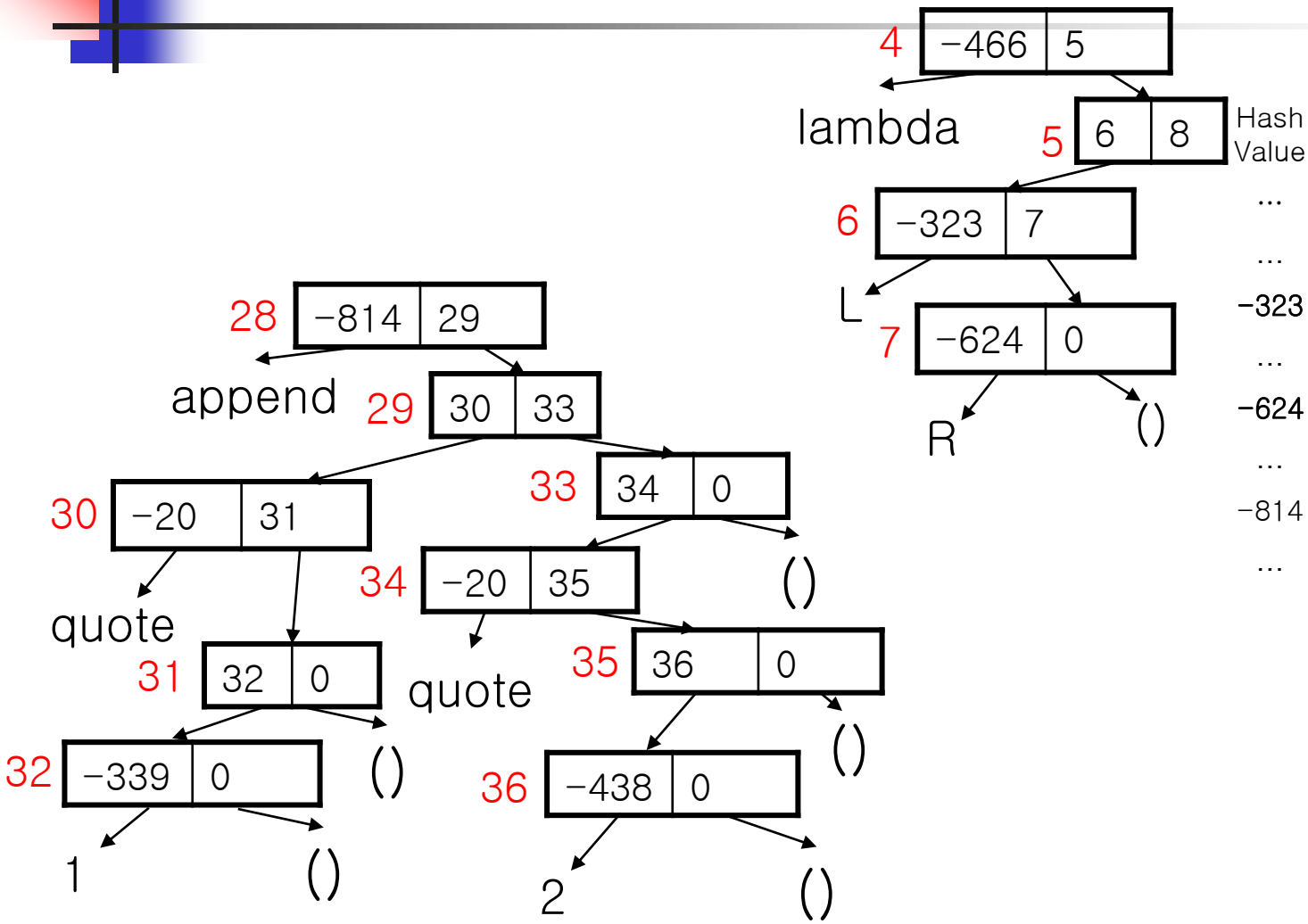


Symbol	Link of Value
L	0
R	36
append	4

Stack

L : 32
R : NULL
L : NULL

Evaluation of (append '(1) '(2))

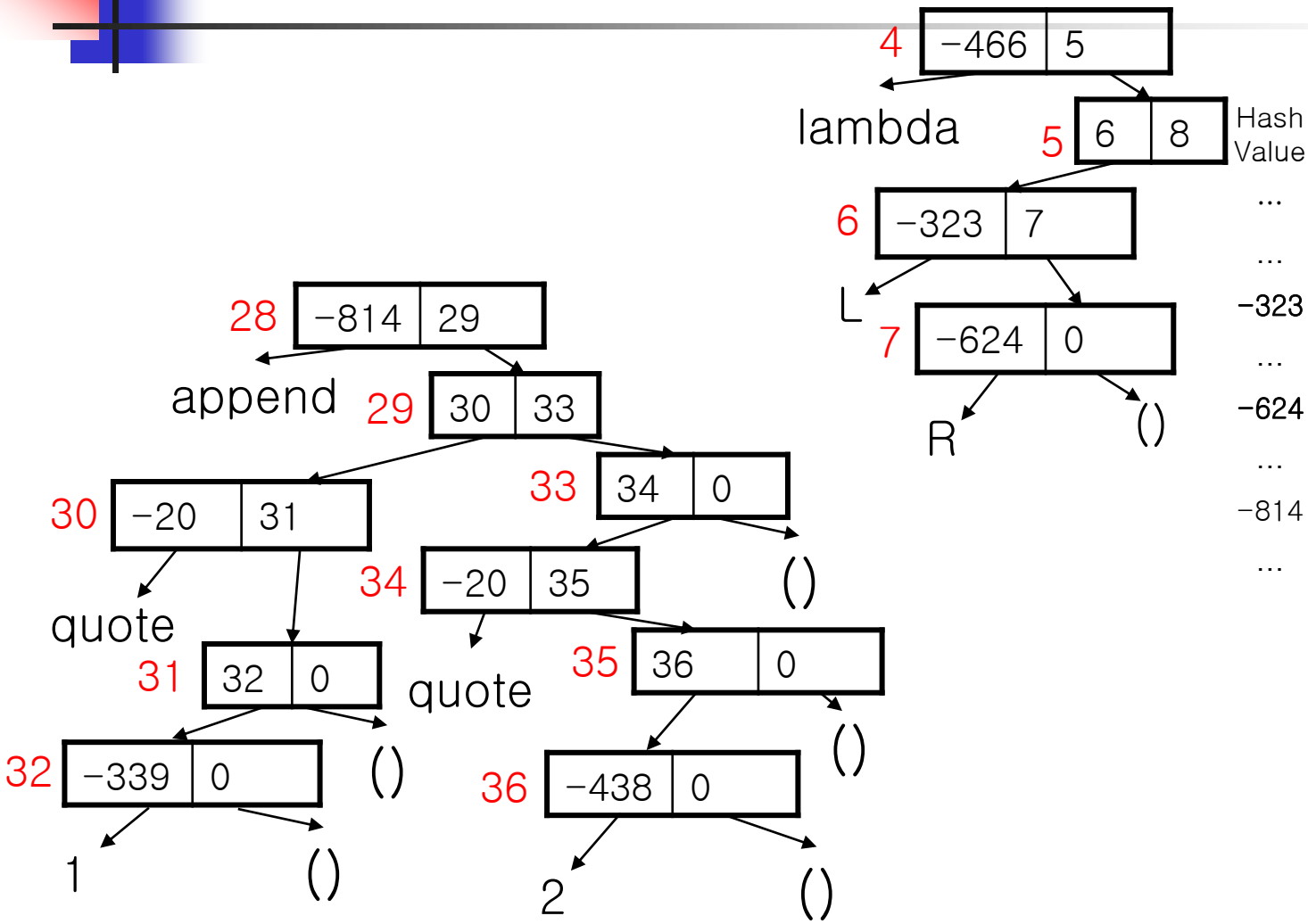


Symbol	Link of Value
L	0
R	36
append	4

Stack

R : 36
L : 32
R : NULL
L : NULL

Evaluation of (append '(1) '(2))

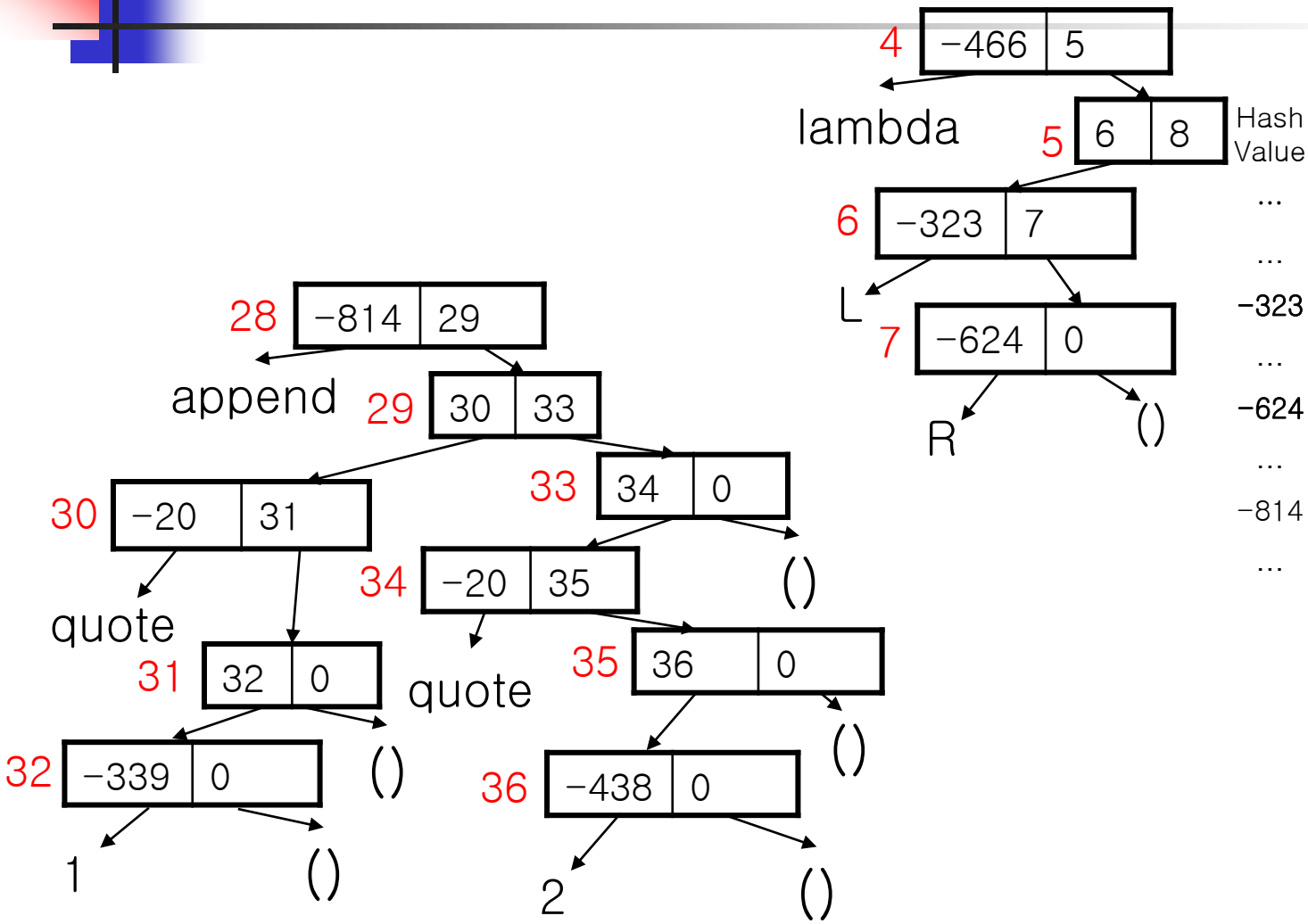


Symbol	Link of Value
L	32
R	36
append	4

Stack

R : NULL
L : NULL

Evaluation of (append '(1) '(2))



Symbol	Link of Value
L	NULL
R	NULL
append	4

Stack